```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

np.random.seed(42)
```

```
n = 200  # number of students

data = {
    "StudyHours": np.random.randint(1, 10, n),
    "Attendance": np.random.randint(50, 100, n),
    "InternalMarks": np.random.randint(40, 100, n),
    "Assignments": np.random.randint(1, 6, n),
    "PreviousGPA": np.round(np.random.uniform(5.0, 10.0, n), 2),
}

df = pd.DataFrame(data)

# Generate FinalMarks using formula + noise
df["FinalMarks"] = (
    0.3 * df["StudyHours"] +
    0.2 * df["Attendance"] +
    0.3 * df["InternalMarks"] +
    2 * df["Assignments"] +
    3 * df["PreviousGPA"] +
    np.random.normal(0, 5, n)
)

df["FinalMarks"] = df["FinalMarks"].round(2)

df.head()
```

	StudyHours	Attendance	InternalMarks	Assignments	PreviousGPA	FinalMarks
0	7	50	48	2	9.75	57.61
1	4	60	51	2	5.74	46.96
2	8	77	92	2	9.63	83.11
3	5	74	40	5	7.46	65.55
4	7	99	97	3	6.29	82.80

Next steps: Generate code with df New interactive sheet

```
print("Dataset Shape:", df.shape)
print("\nMissing Values:")
print(df.isnull().sum())
```

```
Dataset Shape: (200, 6)

Missing Values:
StudyHours       0
Attendance       0
InternalMarks    0
Assignments      0
PreviousGPA      0
FinalMarks       0
dtype: int64
```

```
X = df.drop("FinalMarks", axis=1)
y = df["FinalMarks"]

X.head()
```

	StudyHours	Attendance	InternalMarks	Assignments	PreviousGPA
0	7	50	48	2	9.75
1	4	60	51	2	5.74
2	8	77	92	2	9.63
3	5	74	40	5	7.46
4	7	99	97	3	6.29

Next steps: Generate code with X New interactive sheet

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_scaled[:5]
```

```
array([[ 0.73446706, -1.89597162, -1.30422897, -0.68985793,  1.73941854],
       [-0.40719158, -1.17807249, -1.12589319, -0.68985793, -1.11488777],
       [ 1.11501994,  0.04235605,  1.3113624 , -0.68985793,  1.65400289],
       [-0.0266387 , -0.17301369, -1.77979103,  1.34913102,  0.10940321],
       [ 0.73446706,  1.62173415,  1.60858869, -0.01019494, -0.72339938]])
```


```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_scaled[:5]
```

```
array([[ 0.73446706, -1.89597162, -1.30422897, -0.68985793,  1.73941854],
       [-0.40719158, -1.17807249, -1.12589319, -0.68985793, -1.11488777],
       [ 1.11501994,  0.04235605,  1.3113624 , -0.68985793,  1.65400289],
       [-0.0266387 , -0.17301369, -1.77979103,  1.34913102,  0.10940321],
       [ 0.73446706,  1.62173415,  1.60858869, -0.01019494, -0.72339938]])
```


```
model = LinearRegression()
model.fit(X_train, y_train)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipykernel_10464/1905445833.py in <cell line: 0>()
      1 model = LinearRegression()
----> 2 model.fit(X_train, y_train)

NameError: name 'X_train' is not defined
```

Next steps: Explain error

```
y_pred = model.predict(X_test)

y_pred[:5]
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipykernel_10464/2769633674.py in <cell line: 0>()
----> 1 y_pred = model.predict(X_test)
      2
      3 y_pred[:5]

NameError: name 'X_test' is not defined
```

Next steps: Explain error

```
print("Mean Absolute Error (MAE):", mean_absolute_error(y_test, y_pred))
print("Mean Squared Error (MSE):", mean_squared_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipykernel_10464/3062896874.py in <cell line: 0>()
----> 1 print("Mean Absolute Error (MAE):", mean_absolute_error(y_test, y_pred))
      2 print("Mean Squared Error (MSE):", mean_squared_error(y_test, y_pred))
      3 print("R2 Score:", r2_score(y_test, y_pred))

NameError: name 'y_test' is not defined
```

Next steps: Explain error

Start coding or generate with AI.

```
# ============================================================
#   Customer Segmentation using K-Means Clustering
#   Google Colab Version — Copy each cell block separately
# ============================================================


# ────────────────────────────────────────────────────────────
# CELL 1 — Install & Import Libraries
# ────────────────────────────────────────────────────────────

!pip install scikit-learn pandas matplotlib seaborn -q

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score

print("✅ All libraries imported successfully!")


# ────────────────────────────────────────────────────────────
# CELL 2 — Create Sample Dataset
# ────────────────────────────────────────────────────────────

np.random.seed(42)
n = 200

data = pd.DataFrame({
    "CustomerID":    range(1, n + 1),
    "Age":           np.random.randint(18, 70, n),
    "AnnualIncome":  np.random.randint(15, 120, n),   # in $1000s
    "SpendingScore": np.random.randint(1, 100, n),    # 1-100 scale
})

print("── Dataset Preview ──")
print(data.head(10))
print(f"\nShape: {data.shape}")
print(f"\nBasic Statistics:")
data.describe()


# ────────────────────────────────────────────────────────────
# CELL 3 — Exploratory Data Analysis (EDA)
# ────────────────────────────────────────────────────────────

fig, axes = plt.subplots(1, 3, figsize=(15, 4))
fig.suptitle("Feature Distributions", fontsize=14, fontweight="bold")

for ax, col, color in zip(axes,
                          ["Age", "AnnualIncome", "SpendingScore"],
                          ["steelblue", "coral", "mediumseagreen"]):
    sns.histplot(data[col], kde=True, ax=ax, color=color)
    ax.set_title(col, fontsize=12)
    ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

```
# ────────────────────────────────────────────────────────────
# CELL 4 — Pairplot (Feature Relationships)
# ────────────────────────────────────────────────────────────

sns.pairplot(data[["Age", "AnnualIncome", "SpendingScore"]],
             diag_kind="kde",
             plot_kws={"alpha": 0.6, "color": "steelblue"})
plt.suptitle("Pairplot of Features", y=1.02, fontsize=13, fontweight="bold")
plt.show()


# ────────────────────────────────────────────────────────────
# CELL 5 — Preprocessing (Feature Scaling)
# ────────────────────────────────────────────────────────────

features = ["Age", "AnnualIncome", "SpendingScore"]
X = data[features].values

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

print("✅ Features scaled using StandardScaler")
print(f"\nOriginal Data Sample:\n{X[:3]}")
print(f"\nScaled Data Sample:\n{X_scaled[:3].round(3)}")


# ────────────────────────────────────────────────────────────
# CELL 6 — Find Optimal K (Elbow Method + Silhouette Score)
# ────────────────────────────────────────────────────────────

inertias, silhouettes = [], []
k_range = range(2, 11)

for k in k_range:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = km.fit_predict(X_scaled)
    inertias.append(km.inertia_)
    silhouettes.append(silhouette_score(X_scaled, labels))

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(13, 4))
fig.suptitle("Choosing Optimal Number of Clusters", fontsize=13, fontweight="bold")

ax1.plot(k_range, inertias, "bo-", linewidth=2, markersize=7)
ax1.set_xlabel("Number of Clusters (k)")
ax1.set_ylabel("Inertia")
ax1.set_title("Elbow Method")
ax1.grid(True, alpha=0.3)

ax2.plot(k_range, silhouettes, "ro-", linewidth=2, markersize=7)
ax2.set_xlabel("Number of Clusters (k)")
ax2.set_ylabel("Silhouette Score")
ax2.set_title("Silhouette Score (Higher = Better)")
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

best_k = k_range[np.argmax(silhouettes)]
print(f"\n✅ Best k by Silhouette Score: {best_k}")


# ────────────────────────────────────────────────────────────
# CELL 7 — Train Final K-Means Model
# ────────────────────────────────────────────────────────────

kmeans = KMeans(n_clusters=best_k, random_state=42, n_init=10)
data["Cluster"] = kmeans.fit_predict(X_scaled)

print(f"✅ K-Means model trained with k={best_k}")
print(f"\n── Cluster Sizes ──")
print(data["Cluster"].value_counts().sort_index())

print(f"\n── Cluster Means ──")
data.groupby("Cluster")[features].mean().round(1)


# ────────────────────────────────────────────────────────────
```

```
# CELL 8 — Visualize Clusters (Scatter Plots)
# ────────────────────────────────────────────────────────────────

palette = sns.color_palette("Set2", best_k)

fig, axes = plt.subplots(1, 3, figsize=(17, 5))
fig.suptitle(f"Customer Segments (k={best_k})", fontsize=14, fontweight="bold")

pairs = [
    ("AnnualIncome", "SpendingScore"),
    ("Age",          "SpendingScore"),
    ("Age",          "AnnualIncome"),
]

for ax, (x_col, y_col) in zip(axes, pairs):
    sns.scatterplot(
        data=data, x=x_col, y=y_col,
        hue="Cluster", palette=palette,
        s=80, alpha=0.8, ax=ax,
        legend=(ax == axes[0])
    )
    ax.set_title(f"{x_col} vs {y_col}", fontsize=11)
    ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()


# ────────────────────────────────────────────────────────────────
# CELL 9 — Segment Profiling (Summary Table)
# ────────────────────────────────────────────────────────────────

print("── Segment Profiles ──\n")
for cluster_id in sorted(data["Cluster"].unique()):
    subset = data[data["Cluster"] == cluster_id]
    avg = subset[features].mean()
    print(f"Cluster {cluster_id}  (n={len(subset)} customers)")
    print(f"  Avg Age           : {avg['Age']:.1f} years")
    print(f"  Avg Annual Income : ${avg['AnnualIncome']:.1f}k")
    print(f"  Avg Spending Score: {avg['SpendingScore']:.1f} / 100")
    print()

print("✅ Project Complete!")


# ────────────────────────────────────────────────────────────────
# CELL 10 — (BONUS) Predict Cluster for a New Customer
# ────────────────────────────────────────────────────────────────

new_customer = pd.DataFrame({
    "Age":           [35],
    "AnnualIncome":  [70],
    "SpendingScore": [80],
})

new_scaled = scaler.transform(new_customer)
predicted_cluster = kmeans.predict(new_scaled)[0]

print("── New Customer Details ──")
print(new_customer.to_string(index=False))
print(f"\n✅ Predicted Cluster: {predicted_cluster}")
print(f"\nCluster {predicted_cluster} Profile:")
print(data[data["Cluster"] == predicted_cluster][features].mean().round(1))
```